

Ciência da Computação

Aula 7

Estratégia Gulosa

André Luiz Brun



Introdução

- Tipicamente usados para resolver problemas de otimização
- Recebe esse nome pois o algoritmo escolhe, a cada passo, o candidato mais evidente que possa ser adicionado à solução
- Inicialmente existe um conjunto ou lista de candidatos que podem ou não ser empregados



Introdução

- Conforme o algoritmo avança há a formação de dois conjuntos*, um com os candidatos que farão parte da solução e outro de elementos descartados
- **Função para verificação** da compatibilidade da solução. “A solução é possível?”
 - Responsável por garantir o atendimento às restrições
- **Função de seleção**: responsável por determinar quem é selecionado e quem é descartado



Introdução

- **Função objetivo:** fornece o valor da solução encontrada
 - A função de seleção é diretamente relacionada ao objetivo do problema
 - Se o objetivo é maximização ela escolherá o candidato que proporcione o maior ganho individual
- **O algoritmo nunca muda de ideia** (não desfaz escolhas)



Introdução

Entrada: C : conjunto de possibilidades a serem escolhidas

Saída: C' : conjunto de opções selecionadas a partir de C

- 1: **while** Critério de parada não for satisfeito **do**
 - 2: Selecionar a opção mais interessante
 - 3: **if** Escolha é possível **then**
 - 4: Atualizar o conjunto de saída adicionando a escolha
 - 5: **else**
 - 6: Descartar a opção selecionada
 - 7: **end if**
 - 8: **end while**
 - 9: **return** Conjunto formado
-



Introdução

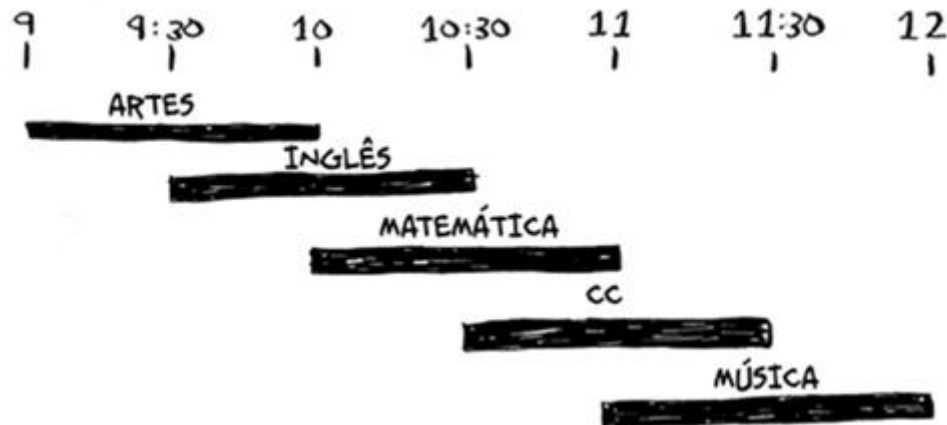
- **Função objetivo:** assistir ao maior número de aulas

AULA	INÍCIO	FIM
ARTES	9 AM	10 AM
INGLÊS	9:30 AM	10:30 AM
MATEMÁTICA	10 AM	11 AM
CC	10:30 AM	11:30 AM
MÚSICA	11 AM	12 PM



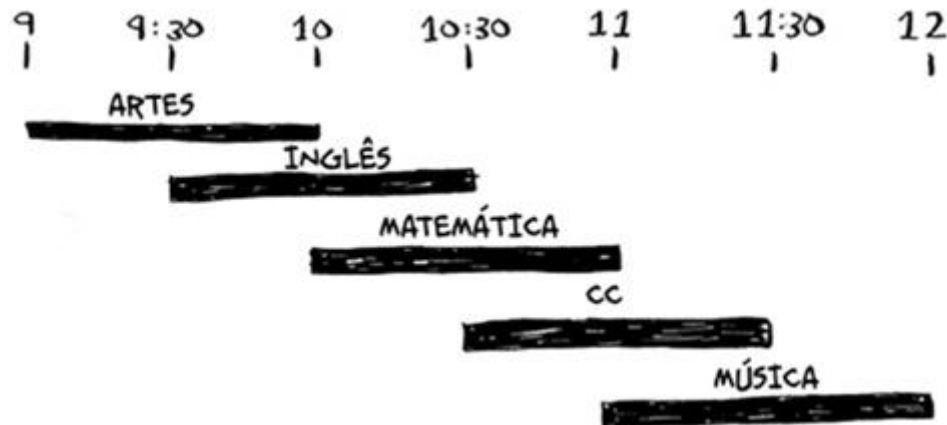
Introdução

- **Restrição:** não pode assistir a duas aulas ao mesmo tempo



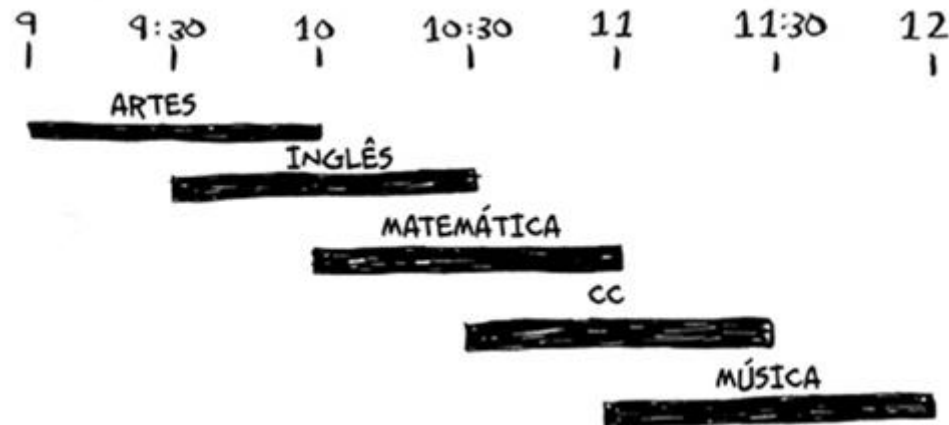
Introdução

- **Seleção:** escolhe a aula que tem o menor tempo final



Introdução

- **Saída:** 3 aulas (Artes, Matemática e Música)



Mochila Binária

Entrada: S : conjunto de n itens disponíveis para serem colocados na mochila

W : vetor contendo o peso W_i de cada um dos n itens presentes no conjunto S

V : vetor contendo o valor V_i de cada um dos n itens presentes no conjunto S

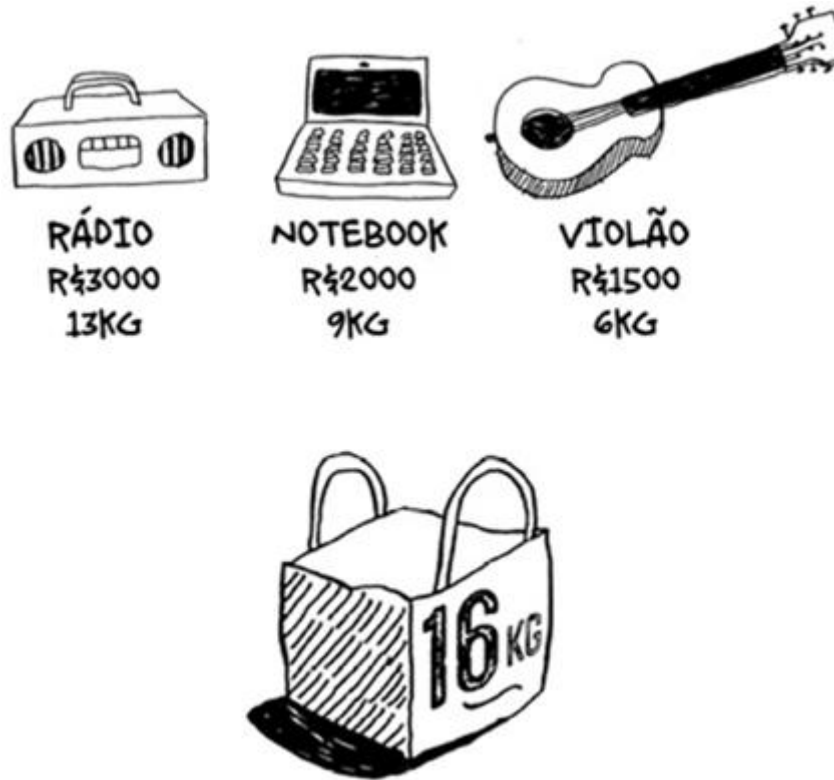
C : capacidade da mochila

Saída: subconjunto de itens selecionados de forma a maximizar o valor presente na mochila

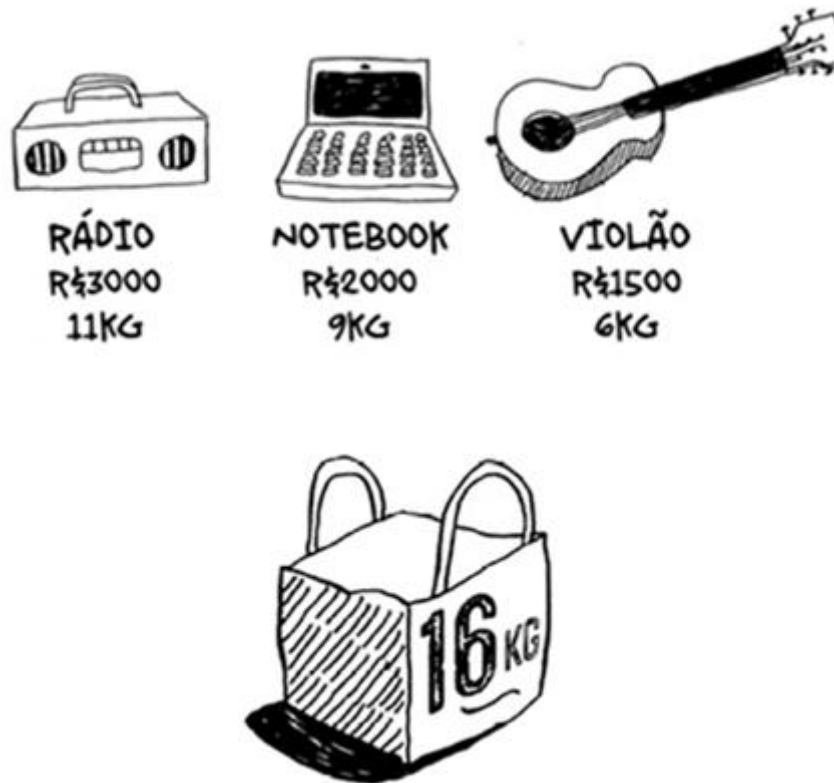
```
1: Mochila = {}
2: c = 0
3: Beneficio = 0
4: for  $i \leftarrow 1$  to  $n$  do
5:      $B_i = V_i / W_i$ 
6: end for
7: while (c < C) e (Ainda houver itens para serem avaliados) do
8:     Selecionar o item  $S_i$  com o maior custo-benefício ( $B_i$ )
9:     if  $W_i \leq (C - c)$  then
10:         Mochila  $\cup S_i$ 
11:         Beneficio +=  $V_i$ 
12:         c +=  $W_i$ 
13:     else
14:         Descarta o item  $S_i$ 
15:     end if
16: end while
17: return Beneficio, Mochila
```



Mochila Binária



Mochila Binária



Mochila Binária

Itens	Custo (R\$)	Benefício
Processador	950	15
Placa mãe	732	25
HD	365	20
Placa de vídeo	1980	45
Memória	156	18
Teclado	170	11
Mouse	75	7
Gabinete	422	13
Fonte	377	19
Monitor	395	16



Mochila Binária

Itens	Custo (R\$)	Benefício	Benefício/Custo
Processador	950	15	0,01579
Placa mãe	732	25	0,03415
HD	365	20	0,05479
Placa de vídeo	1980	45	0,02273
Memória	156	18	0,11538
Teclado	170	11	0,06471
Mouse	75	7	0,09333
Gabinete	422	13	0,03081
Fonte	377	19	0,05040
Monitor	395	16	0,04051



Mochila Binária

Itens	Custo (R\$)	Benefício	Benefício/Custo	Ordem
Processador	950	15	0,01579	10º
Placa mãe	732	25	0,03415	7º
HD	365	20	0,05479	4º
Placa de vídeo	1980	45	0,02273	8º
Memória	156	18	0,11538	1º
Teclado	170	11	0,06471	3º
Mouse	75	7	0,09333	2º
Gabinete	422	13	0,03081	9º
Fonte	377	19	0,05040	5º
Monitor	395	16	0,04051	6º



Mochila Binária

Itens	Custo (R\$)	Benefício	Benefício/Custo	Ordem	Selecionado?
Processador	950	15	0,01579	10º	Não
Placa mãe	732	25	0,03415	7º	Sim
HD	365	20	0,05479	4º	Sim
Placa de vídeo	1980	45	0,02273	8º	Não
Memória	156	18	0,11538	1º	Sim
Teclado	170	11	0,06471	3º	Sim
Mouse	75	7	0,09333	2º	Sim
Gabinete	422	13	0,03081	9º	Sim
Fonte	377	19	0,05040	5º	Sim
Monitor	395	16	0,04051	6º	Sim



Mochila Fracionária

Entrada: S : conjunto de n itens disponíveis para serem colocados na mochila

W : vetor contendo o peso W_i de cada um dos n itens presentes no conjunto S

V : vetor contendo o valor V_i de cada um dos n itens presentes no conjunto S

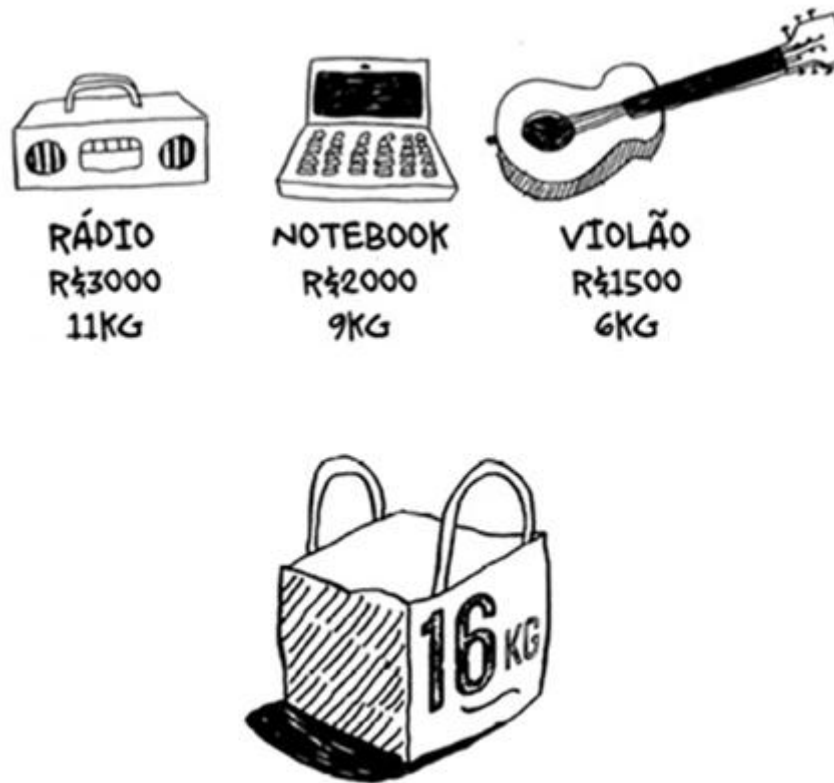
C : capacidade da mochila

Saída: subconjunto de itens selecionados de forma a maximizar o valor presente na mochila

```
1: Mochila = {}
2: c = 0
3: Beneficio = 0
4: for  $i \leftarrow 1$  to  $n$  do
5:      $B_i = V_i / W_i$ 
6: end for
7: while ( $c < C$ ) e (Ainda houver itens para serem avaliados) do
8:     Selecionar o item  $S_i$  com o maior custo-benefício ( $B_i$ )
9:      $a = \min(W_i, C - c)$ 
10:    Mochila  $\cup \frac{a}{W_i} \times S$ 
11:    Beneficio  $+= \frac{a}{W_i} \times V_i$ 
12:     $c += a$ 
13: end while
14: return Beneficio, Mochila
```



Mochila Fracionária



Mochila Fracionária

Itens	Custo (kg)	Benefício
Pão	2	15
Água	7	70
Cerveja	6	40
Carne	3	10
Arroz	3	15
Madeira	7	12
Querosene	1	3
Fósforo	0,2	2



Mochila Fracionária

Itens	Custo (kg)	Benefício	Ben./Custo	Ordem	Quanto Vai?	Ben. Obtido
Pão	2	15	7,50	3º	2,00	15
Água	7	70	10,00	1º	7,00	70
Cerveja	6	40	6,67	4º	6,00	40
Carne	3	10	3,33	6º	1,80	6
Arroz	3	15	5,00	5º	3,00	15
Madeira	7	12	1,71	8º	0,00	0
Querosene	1	3	3,00	7º	0,00	0
Fósforo	0,2	2	10,00	2º	0,20	2
Total					20,00	148



Problema do Pendrive

Tenho n arquivos digitais no meu computador. Cada arquivo i tem t_i megabytes. Quero gravar o maior número possível de arquivos em um pendrive que tem capacidade para c megabytes. O problema pode ser formulado assim: encontrar o maior subconjunto X do intervalo $1..n$ que satisfaça a restrição

$$\sum_{i=1}^n t_i \leq c$$

Exemplo A: A instância do problema do pendrive definida pelos parâmetros $n = 8$, $t = (10, 15, 20, 20, 30, 35, 40, 50)$ e $c = 90$



Escalonamento de Tarefas

$C \rightarrow$ conjunto contendo as tarefas a serem realizadas cada uma com um tempo de início e de término

$m \rightarrow$ saída contendo o número mínimo de máquina necessárias para cumprir as tarefas nos prazos*

*A saída pode ser o escalonamento das tarefas indicando em que máquina cada uma deverá ser executada



Escalonamento de Tarefas

Entrada: C : conjunto de tarefas que devem ser realizadas com seus devidos instantes de início e término

Saída: escalonamento das tarefas de forma a minimizar a quantidade recursos necessário

```
1: while Ainda houver tarefas não designadas do
2:   Selecionar a tarefa com o menor tempo de início
3:   if Há recurso disponível then
4:     Atribuir a tarefa selecionada ao recurso disponível
5:   else
6:     Alocar novo recurso e lhe atribuir a tarefa selecionada
7:   end if
8: end while
9: return Escalonamento das Tarefas
```



Escalonamento de Tarefas

Tarefas	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Início	1	11	13	3	4	5	2	1	7	15	8	14	1	21	13
Término	5	17	23	6	12	12	8	9	11	19	12	17	8	23	27



Escalonamento de Tarefas

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
P1	1	1	1	1	6	6	6	6	6	6	6		3	3	3	3	3	3	3	3	3	3					
P2	13	13	13	13	13	13	13	11	11	11	11		15	15	15	15	15	15	15	15	15	15	15	15	15	15	15
P3	8	8	8	8	8	8	8	8			2	2	2	2	2	2					14	14					
P4		7	7	7	7	7	7							12	12	12											
P5			4	4	4		9	9	9	9					10	10	10	10									
P6				5	5	5	5	5	5	5	5																



Escalonamento de Tarefas (Tempo)

- Nessa aplicação, a ideia é que, dado um conjunto de recursos fixos, as tarefas sejam executadas com o menor tempo possível
- Algoritmo LPT (Longest Processing Time)
 - Alocar as tarefas de maior duração no recurso menos utilizado



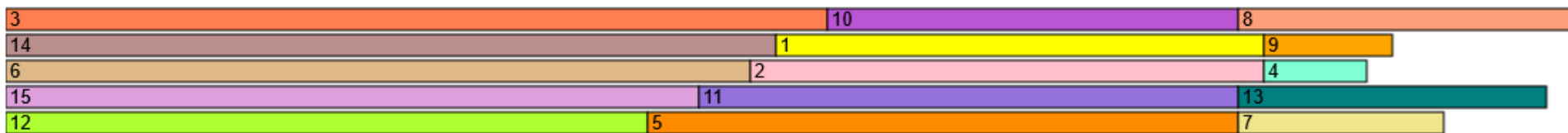
Escalonamento de Tarefas (Tempo)

Tarefas	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Tempo	19	20	32	4	23	29	8	13	5	16	21	25	12	30	27



Escalonamento de Tarefas (Tempo)

Tarefas	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Tempo	19	20	32	4	23	29	8	13	5	16	21	25	12	30	27

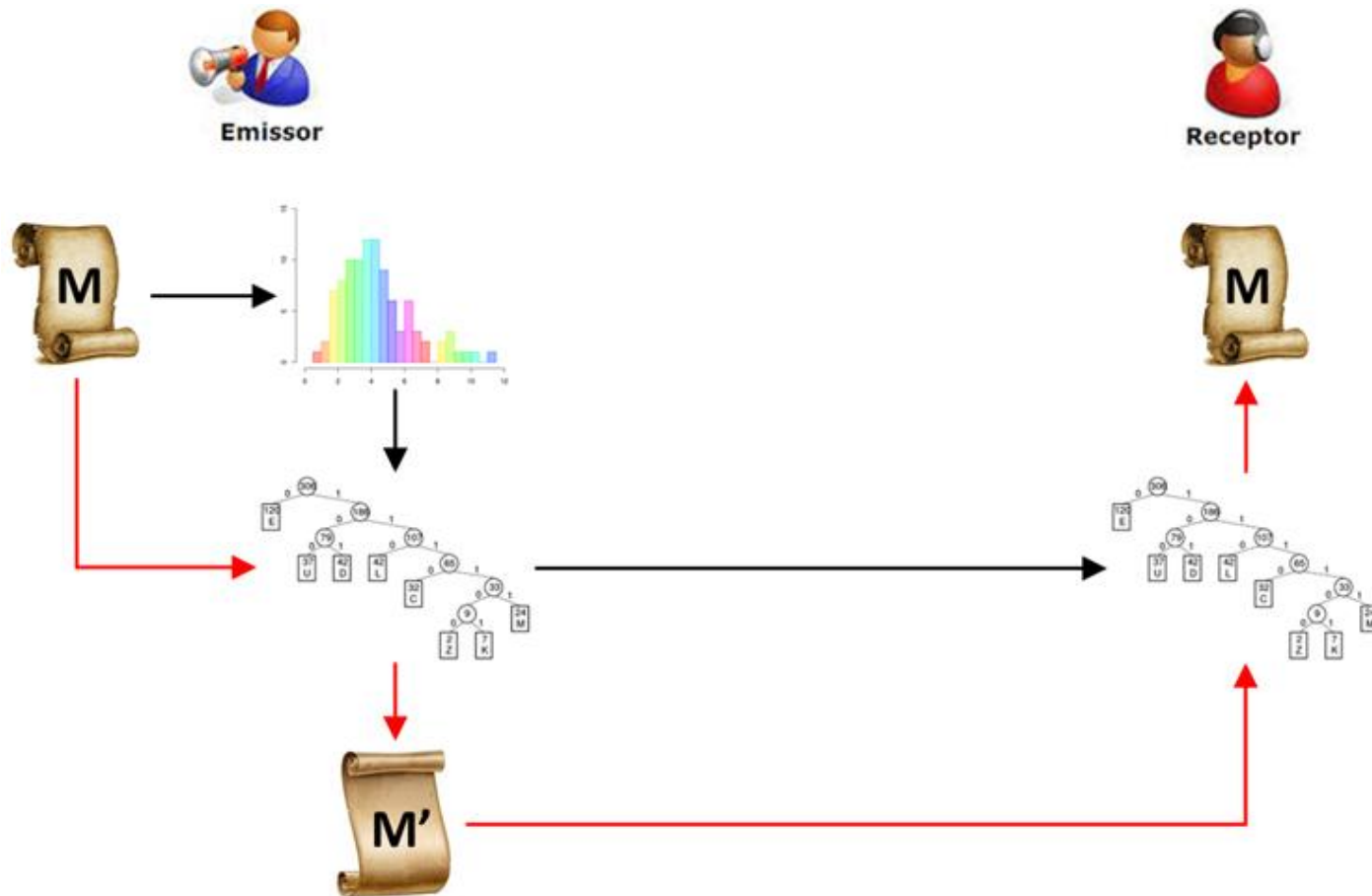


Código de Huffman

- Estratégia para compressão de dados
- Algoritmo guloso para a construção da árvore de Huffman
- Objetivo: economizar o máximo possível no envio da mensagem



Código de Huffman



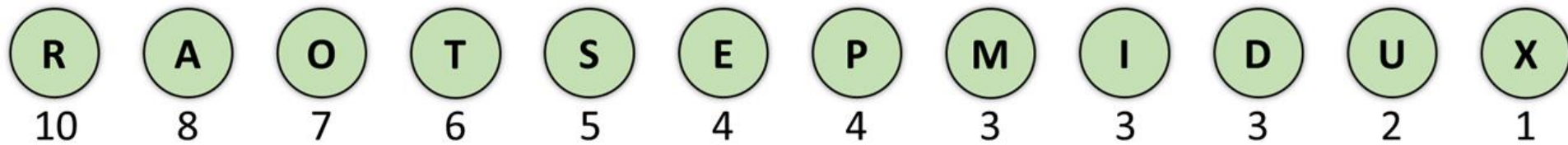
Código de Huffman

“EM RAPIDO RAPTO UM RAPIDO RATO RAPTOU TRES
RATOS SEM DEIXAR RASTROS”

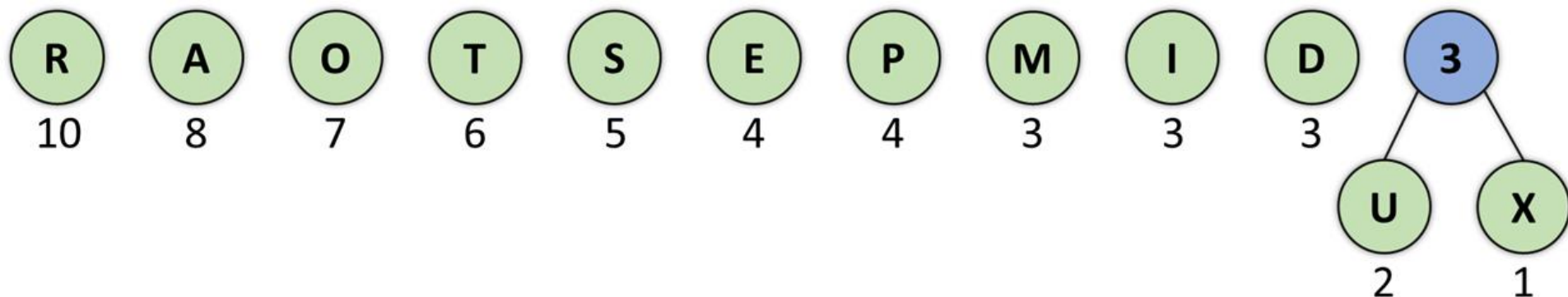


Código de Huffman

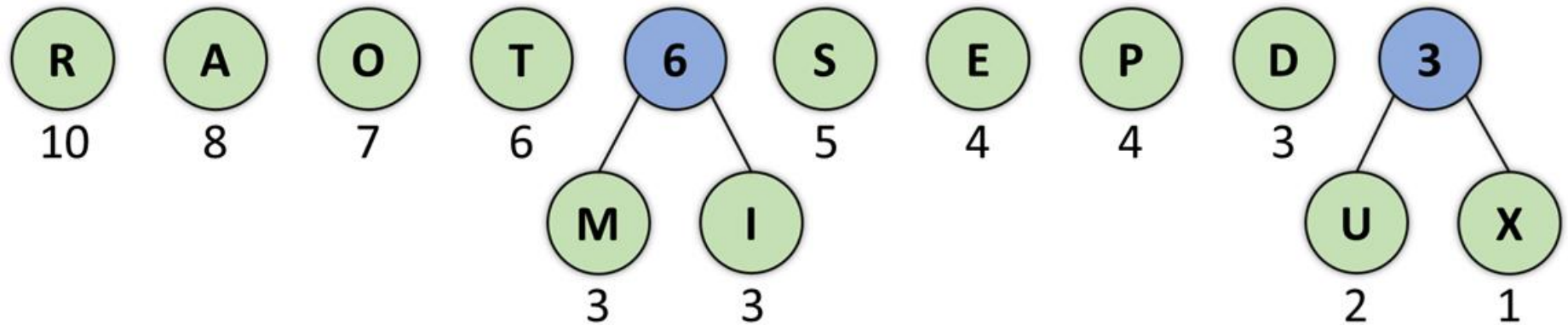
“EM RAPIDO RAPTO UM RAPIDO RATO RAPTOU TRES
RATOS SEM DEIXAR RASTROS”



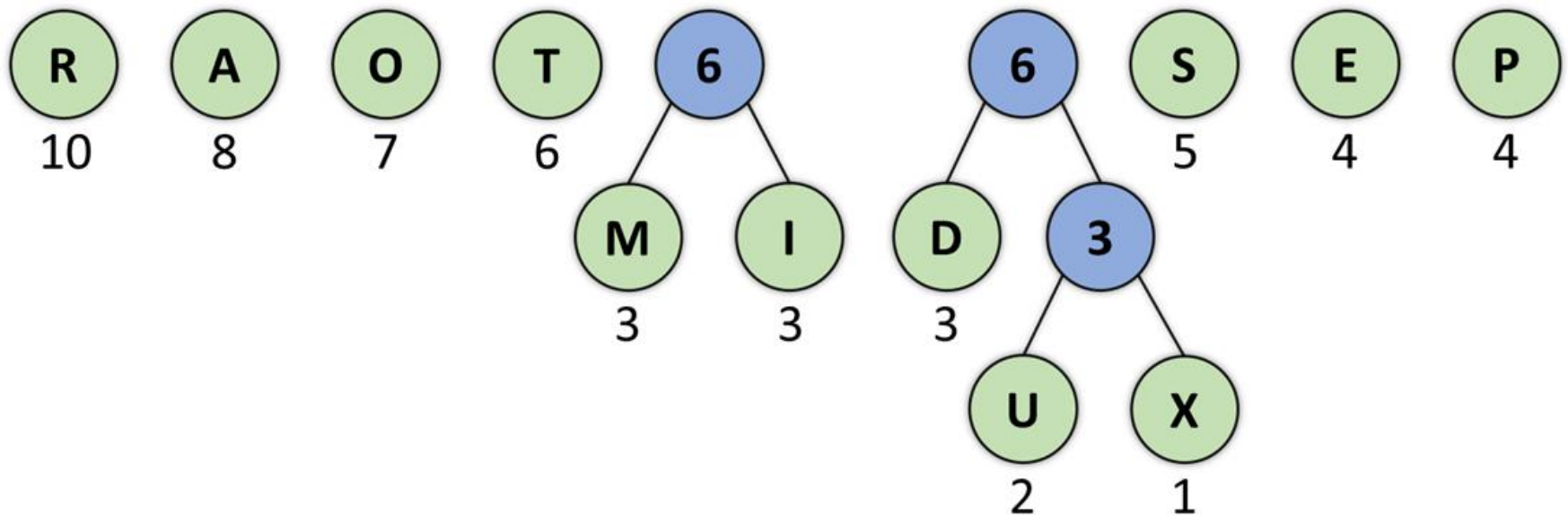
Código de Huffman



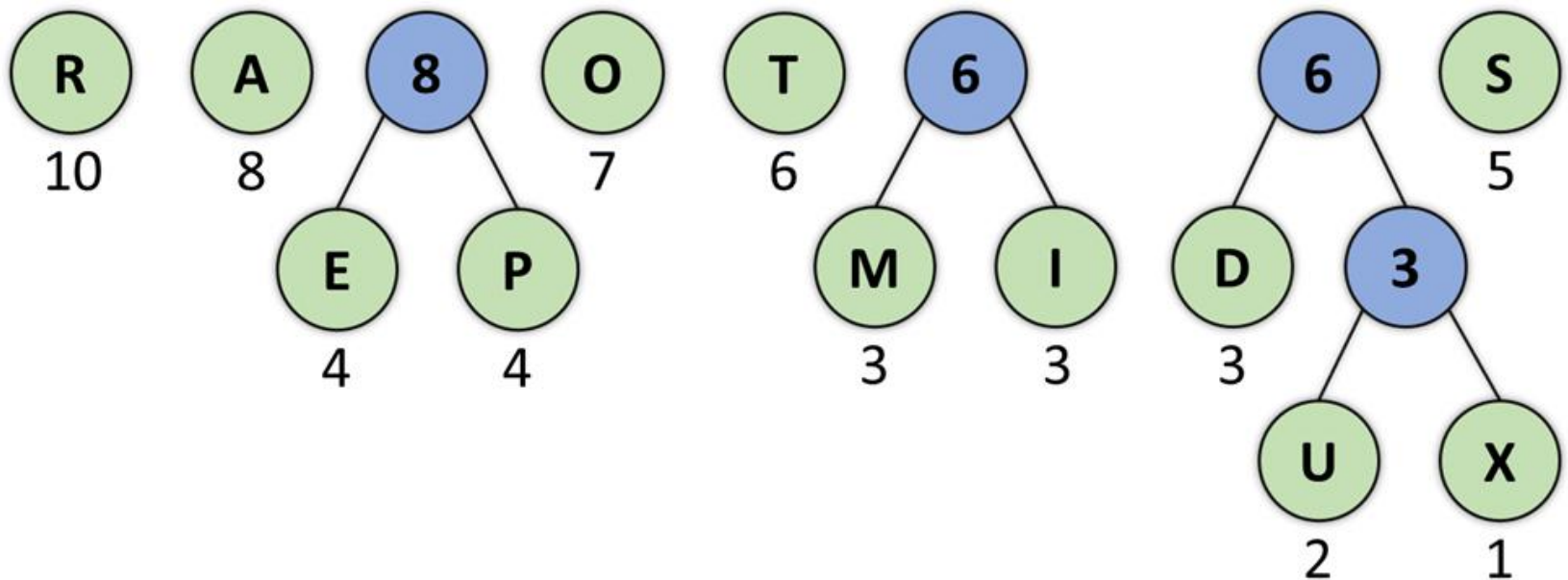
Código de Huffman



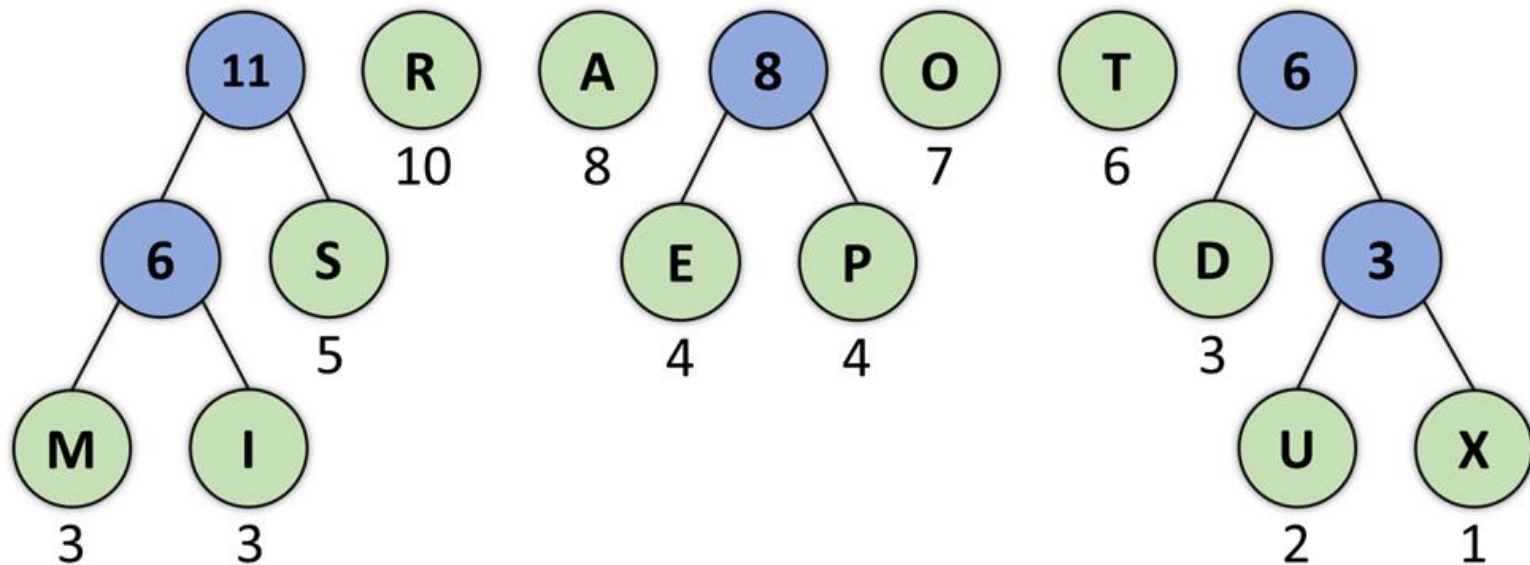
Código de Huffman



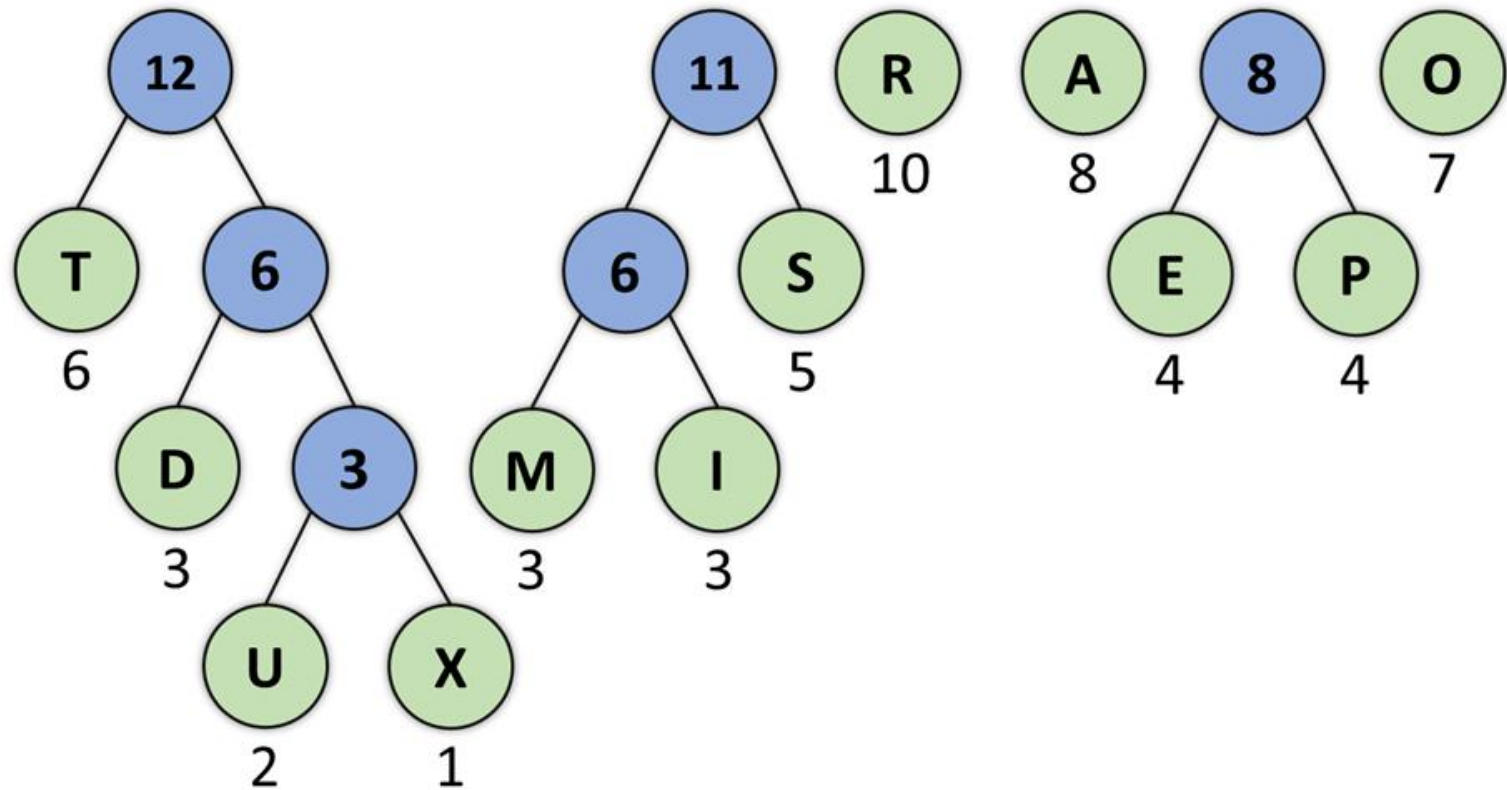
Código de Huffman



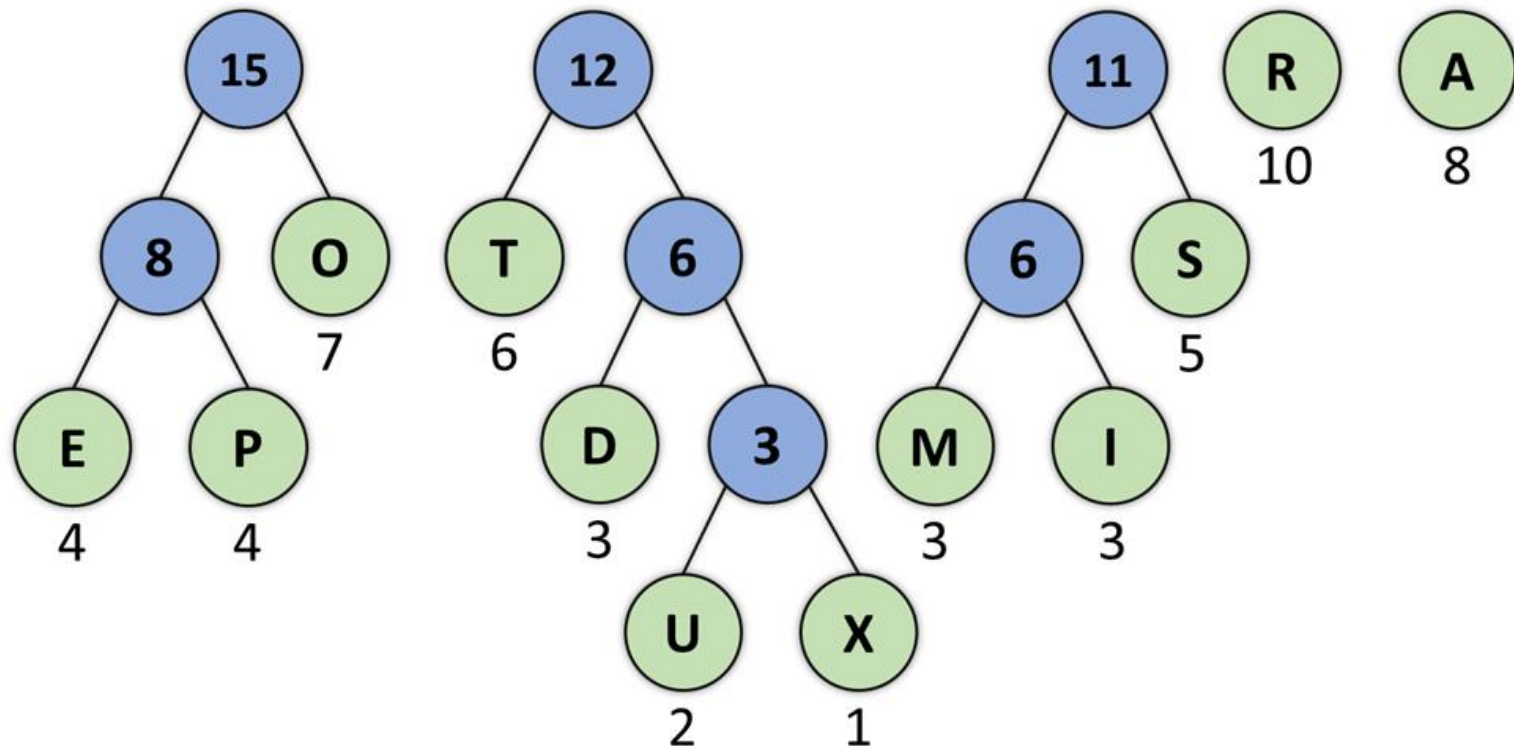
Código de Huffman



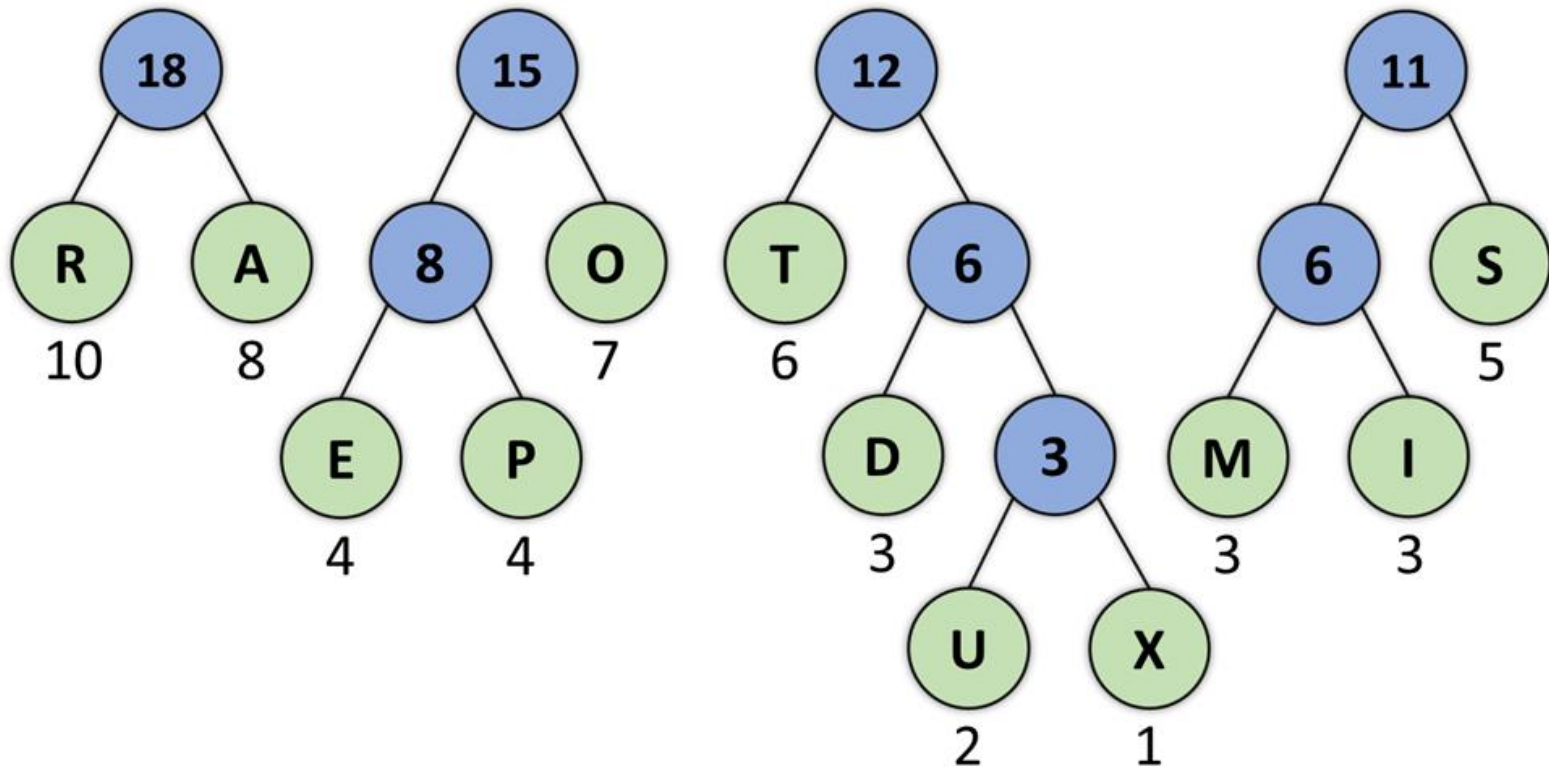
Código de Huffman



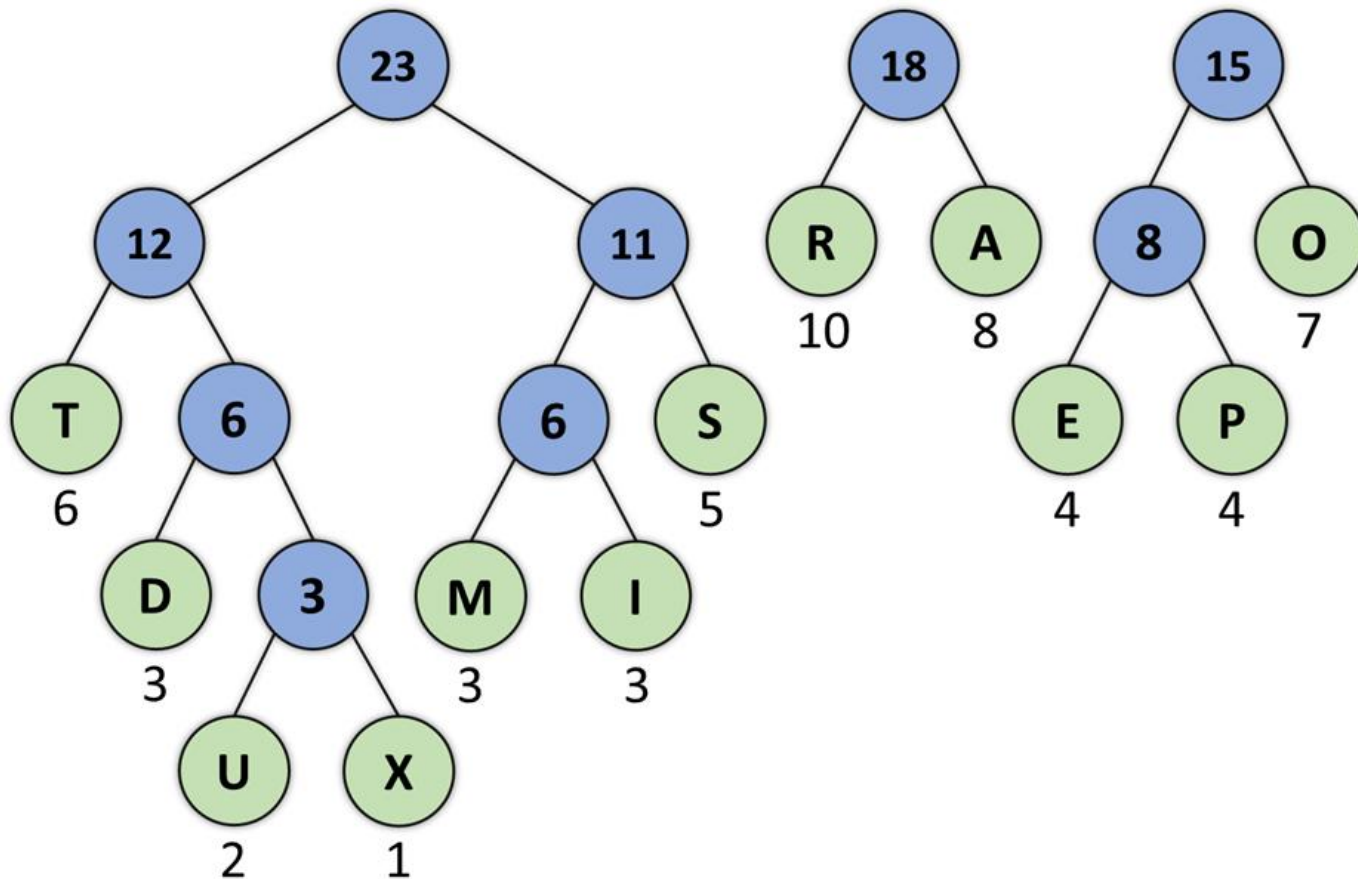
Código de Huffman



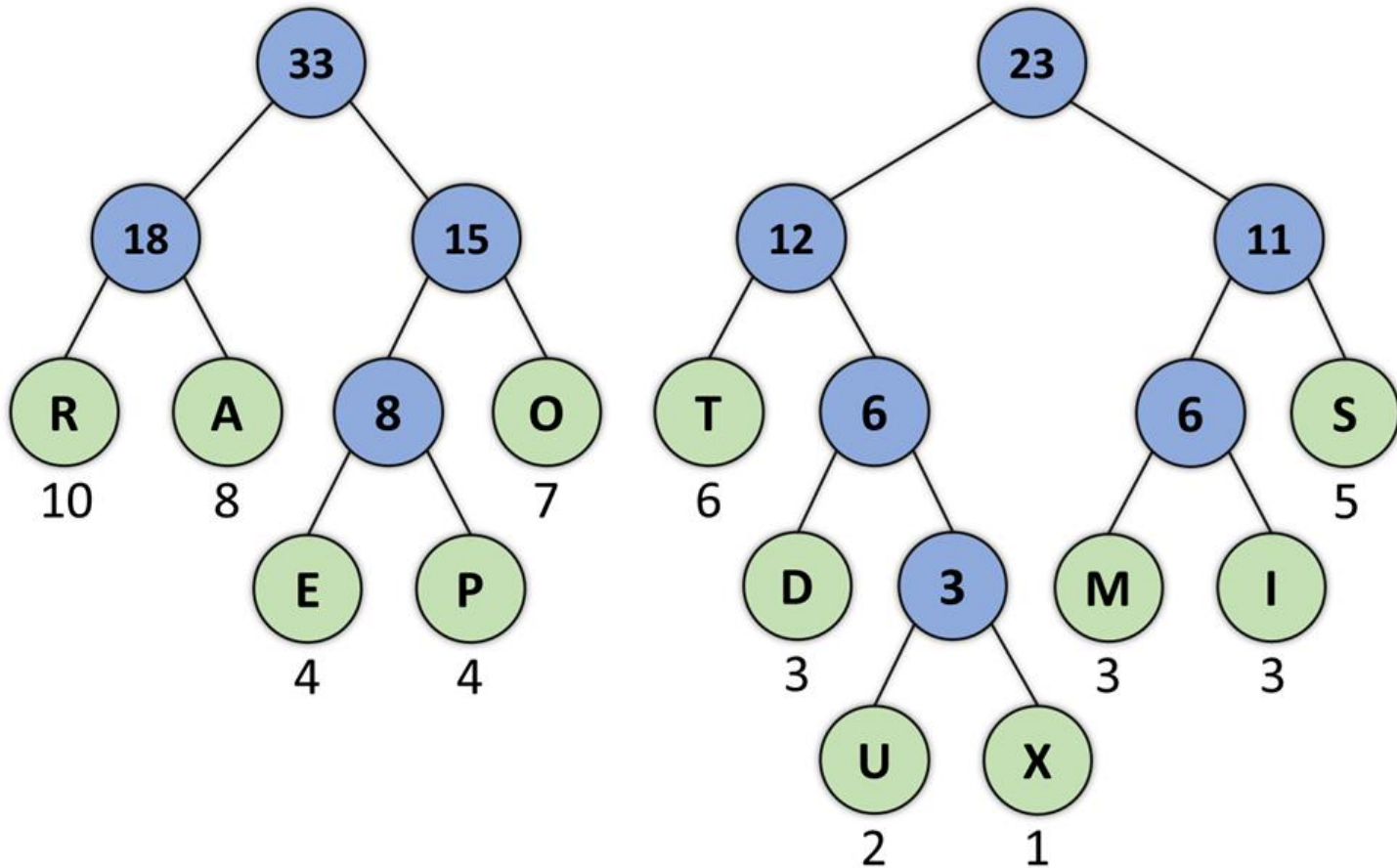
Código de Huffman



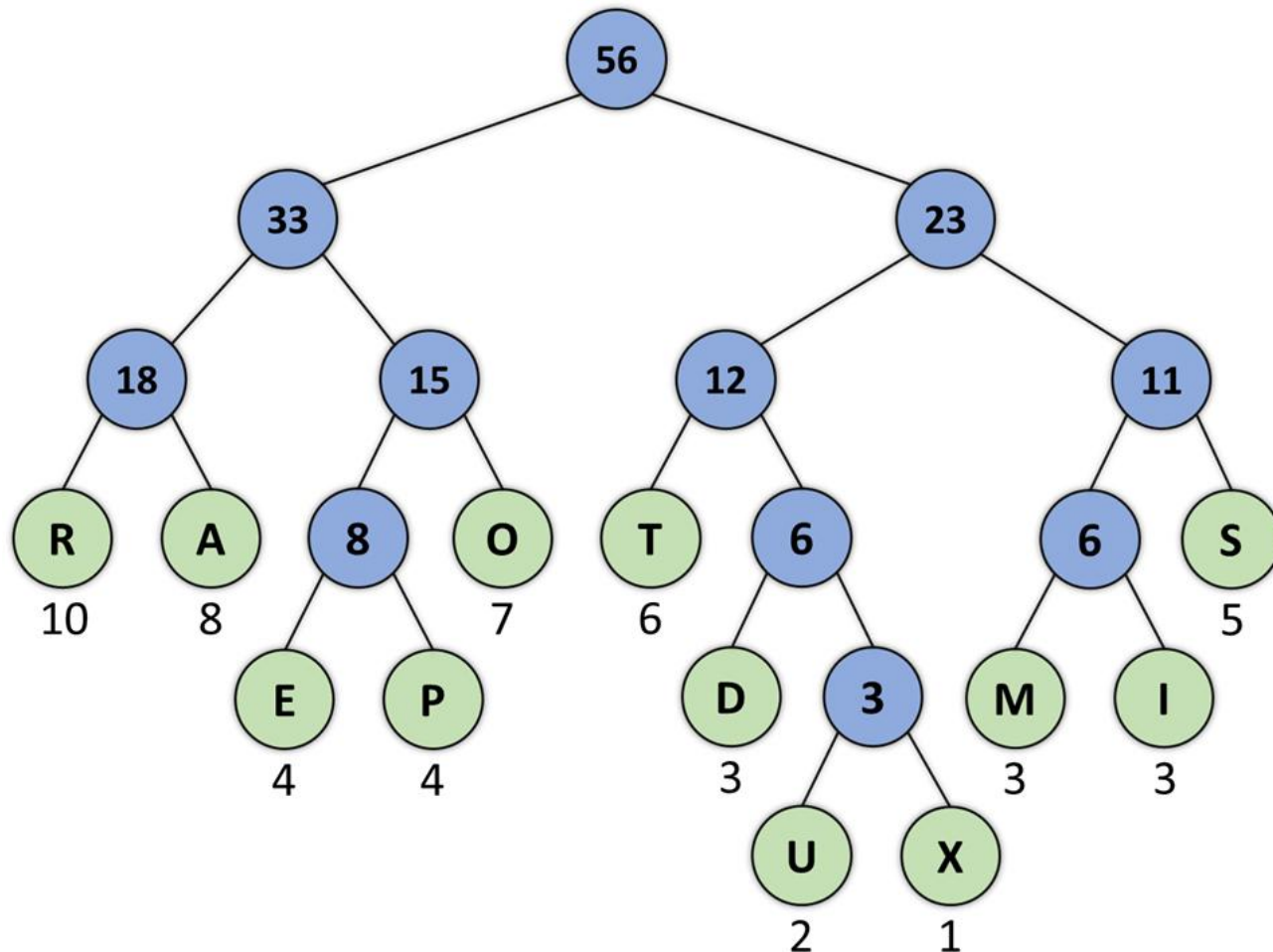
Código de Huffman



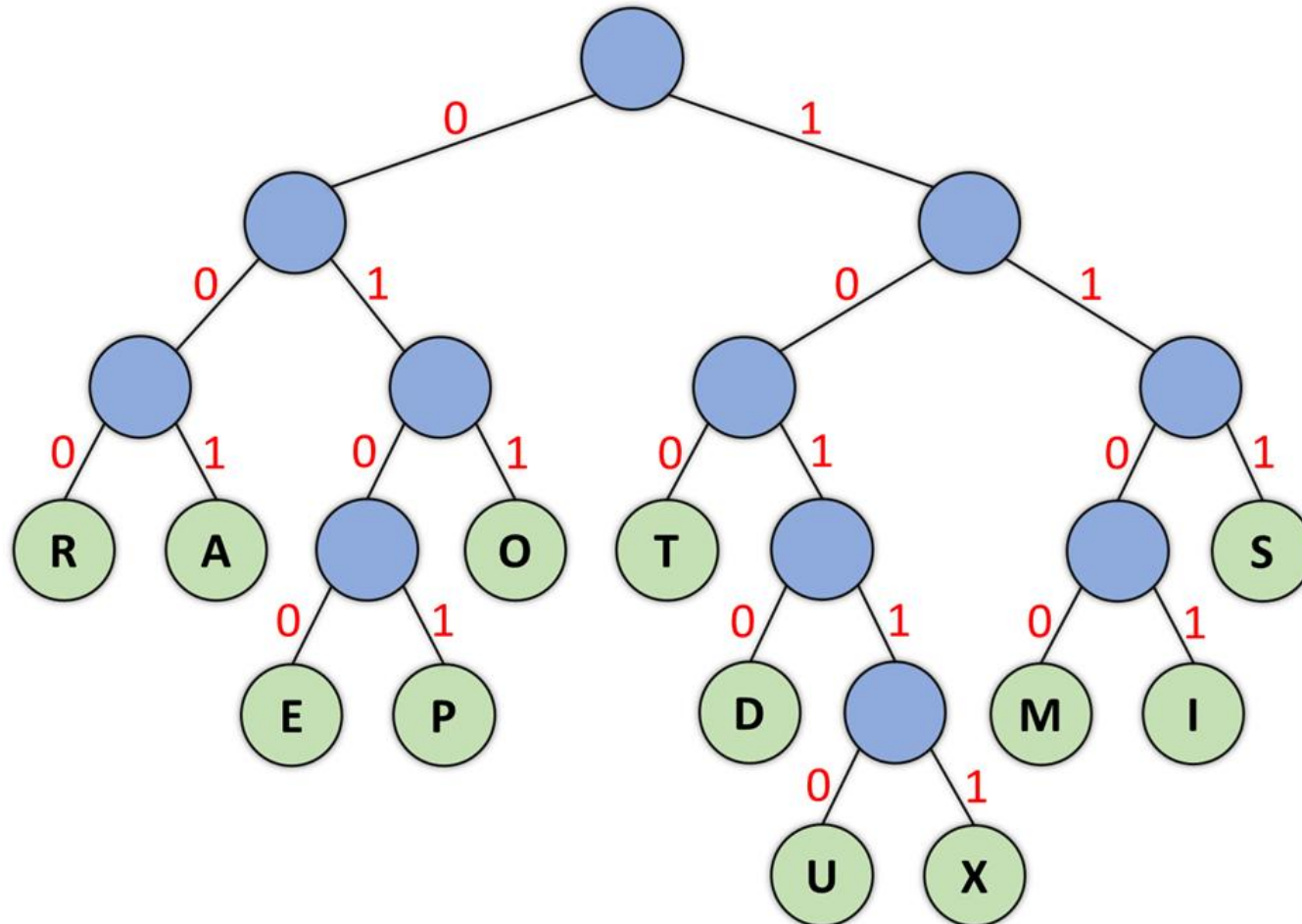
Código de Huffman



Código de Huffman



Código de Huffman

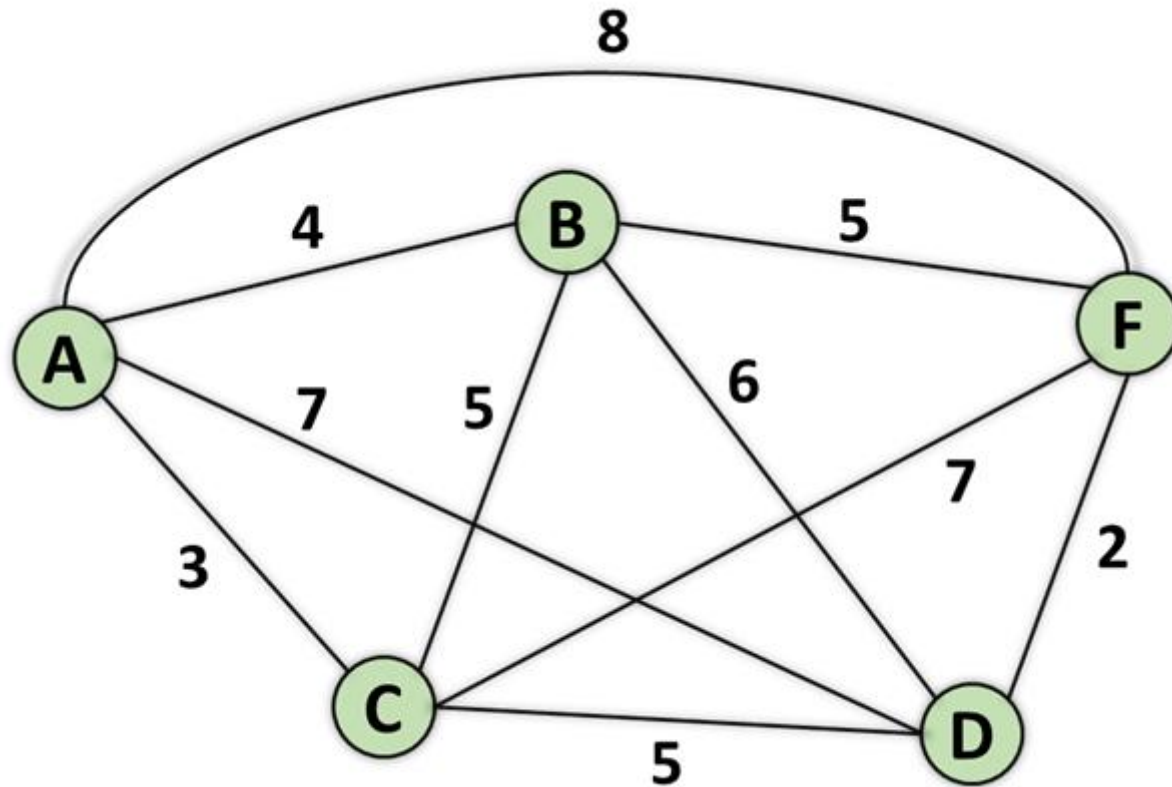


Código de Huffman

Caractere	Frequência	ASCII	Custo ASCII	Huffman	Custo Huffman
E	4	01000101	32	0100	16
M	3	01001101	24	1100	12
R	10	01010010	80	000	30
A	8	01000001	64	001	24
P	4	01010000	32	0101	16
I	3	01001001	24	1101	12
D	3	01001111	24	1010	12
O	7	01000100	56	011	21
T	6	01001111	48	100	18
U	2	01010101	16	10110	10
X	1	01011000	8	10111	5
S	5	01010011	40	111	15
Total	56		448		191



Kruskal



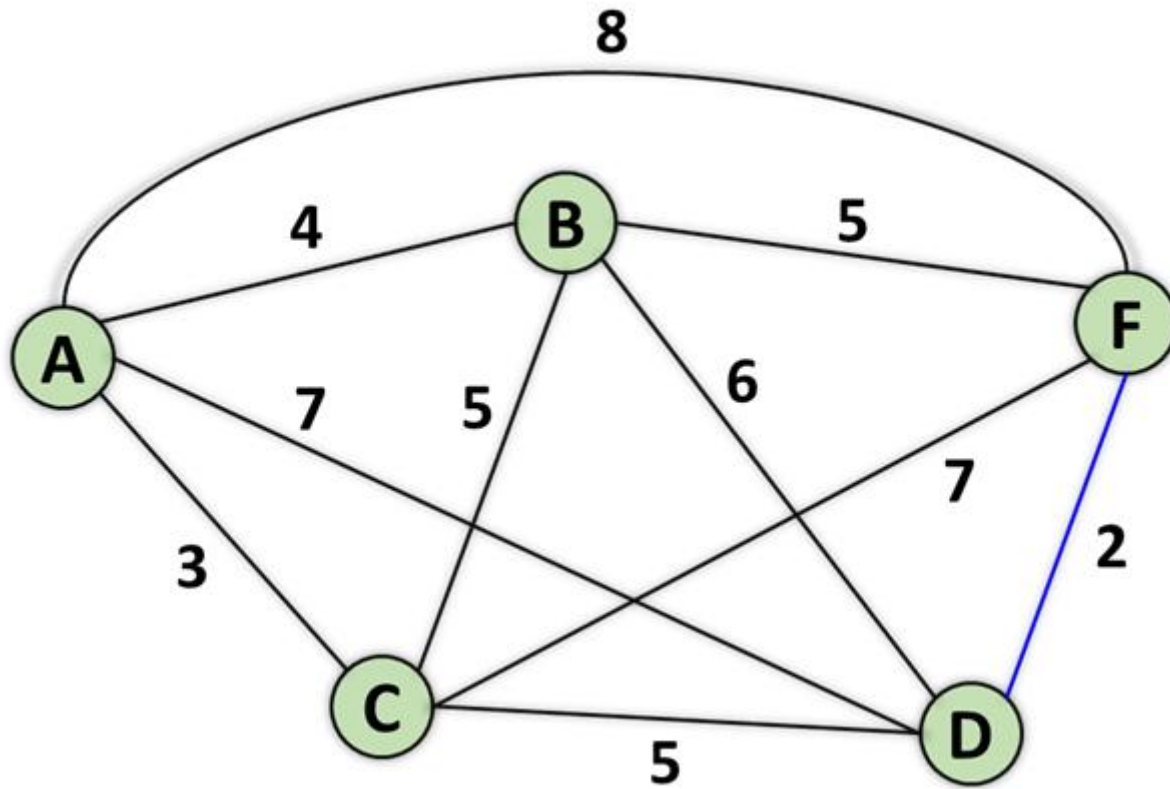
Kruskal

Entrada: G : grafo ponderado e não direcionado composto por V vértices e E arestas
 w : vetor contendo o peso W_i de cada uma das E arestas presentes no grafo G
Saída: conjunto das arestas selecionados para formar a AGM

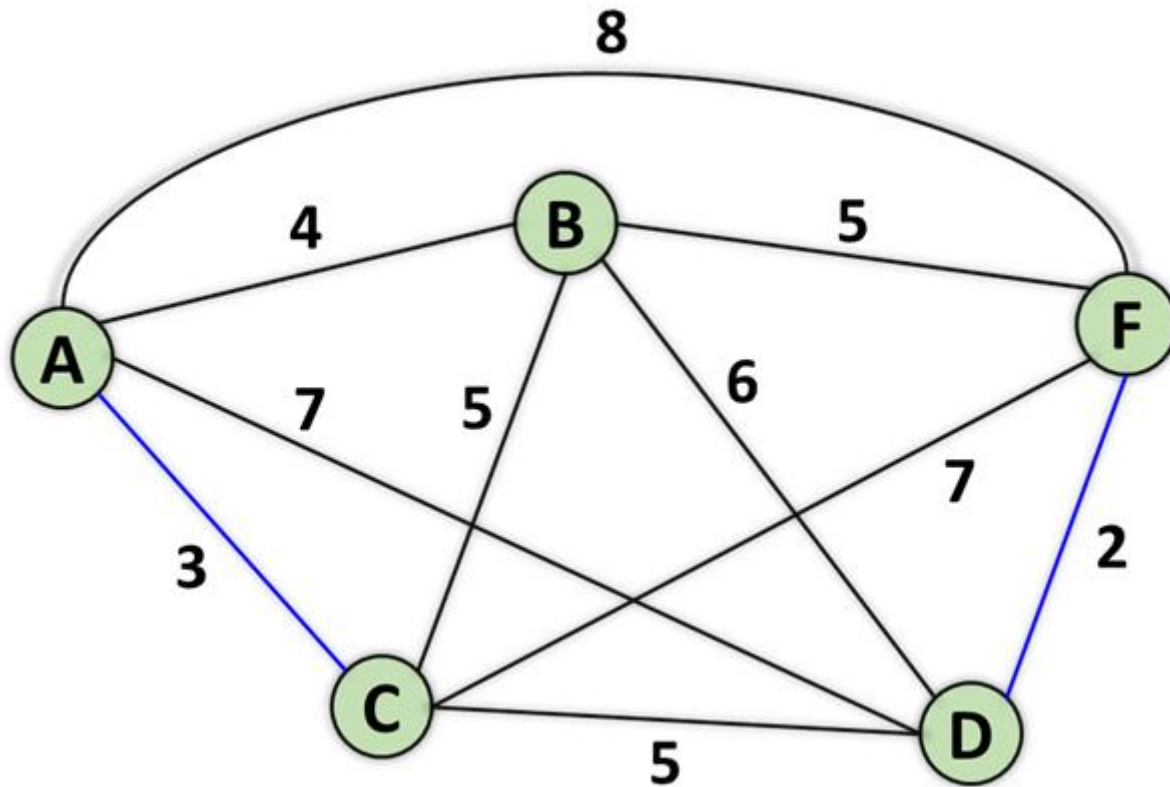
```
1:  $A = \{\}$ 
2: for cada vértice  $v \in G.V$  do
3:   Crie  $(v)$  como um conjunto unitário
4: end for
5:  $c = 0$ 
6: while  $c < V - 1$  do
7:   Selecionar a aresta  $(u, v) \in G.E$  com o menor custo ( $W_i$ )
8:   if  $u$  e  $v$  pertencem ao mesmo conjunto then
9:     Descarta a aresta  $(u, v)$ 
10:  else
11:     $A \cup \{(u, v)\}$ 
12:    Agrupa os conjuntos de  $u$  e de  $v$ 
13:     $++c$ 
14:  end if
15: end while
16: return  $A$ 
```



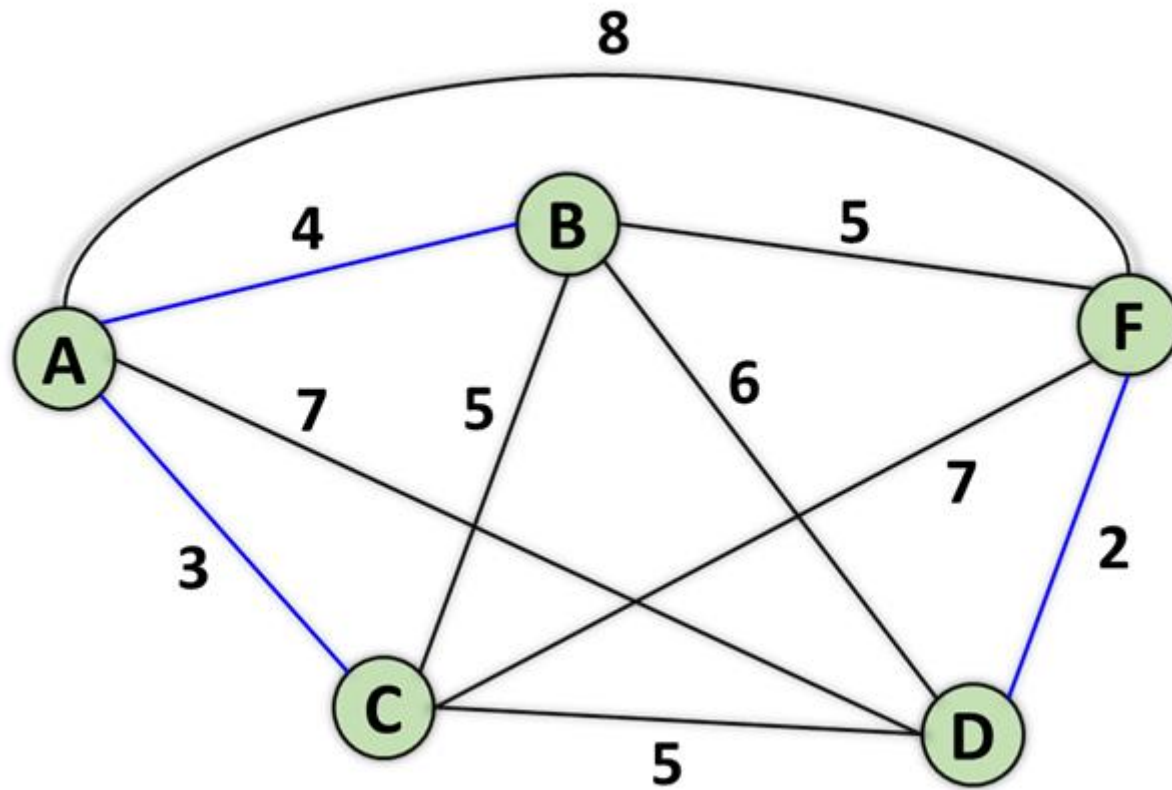
Kruskal



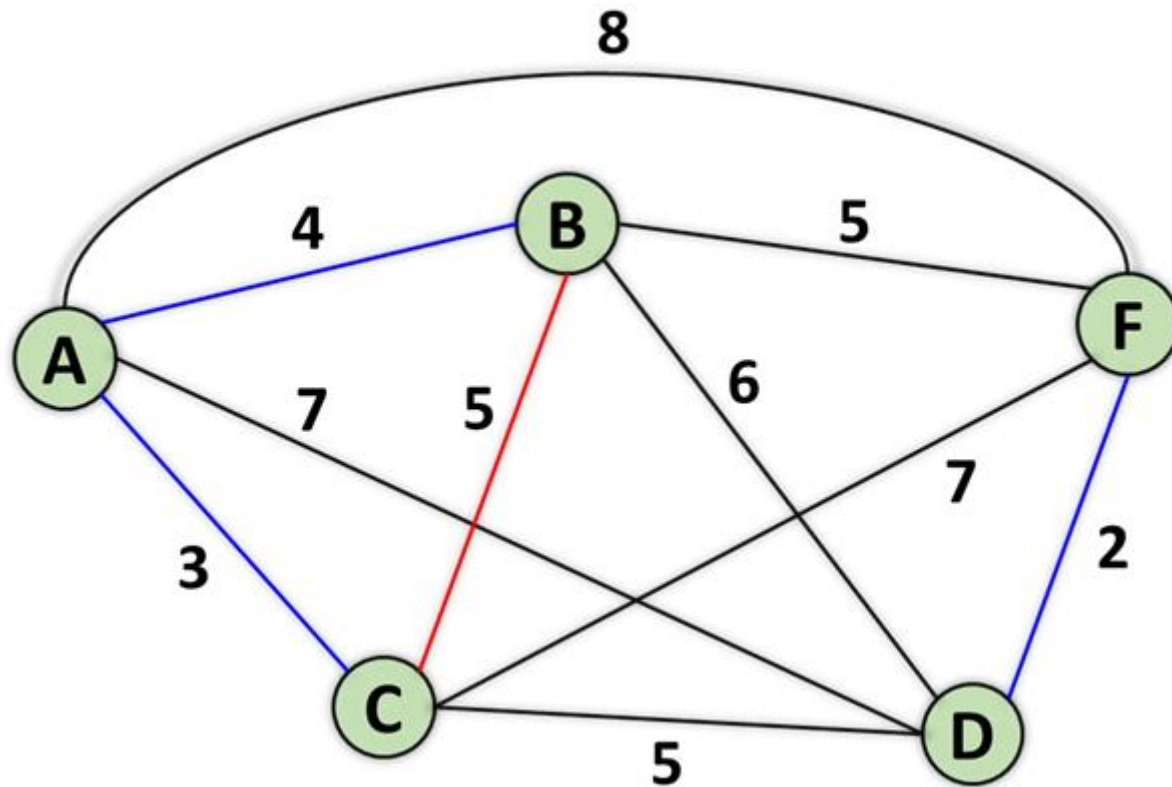
Kruskal



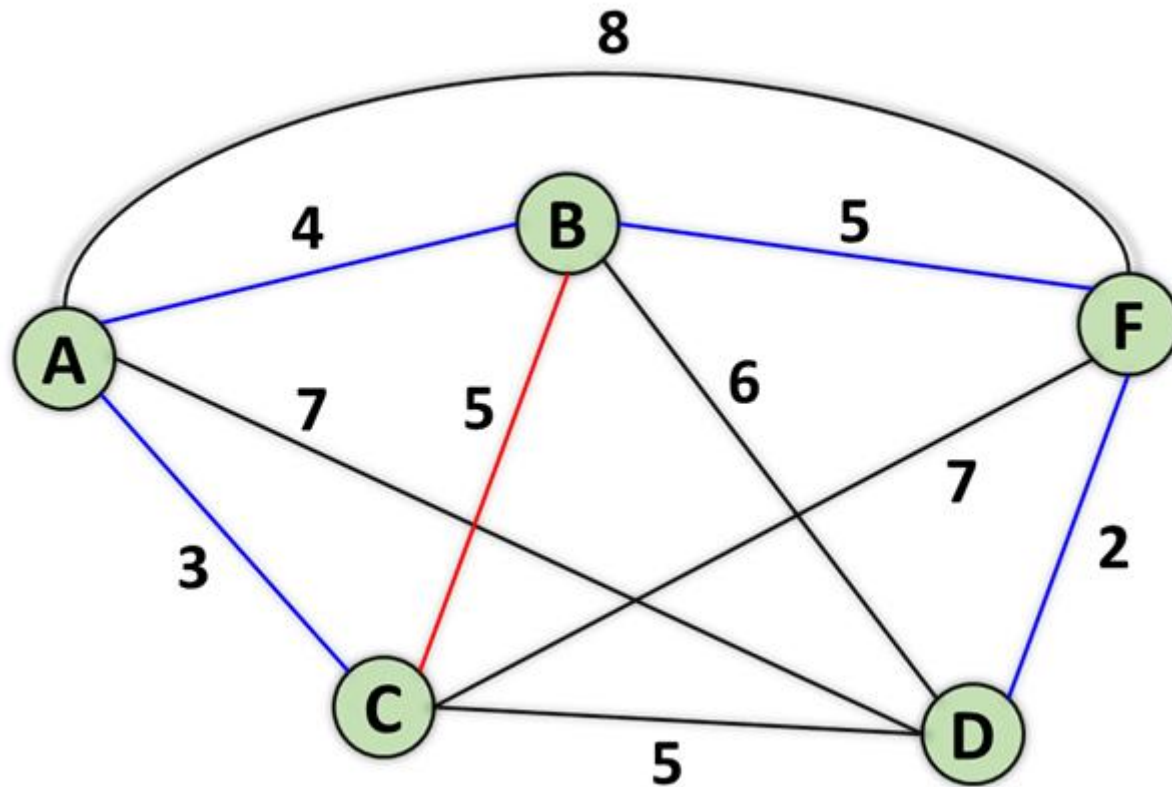
Kruskal



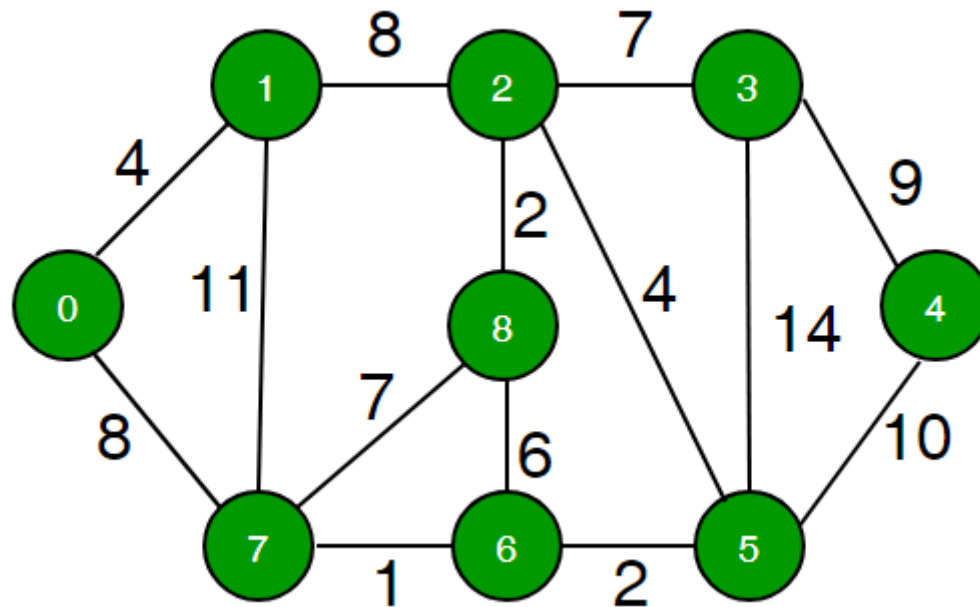
Kruskal



Kruskal



Union-Find



Union-Find

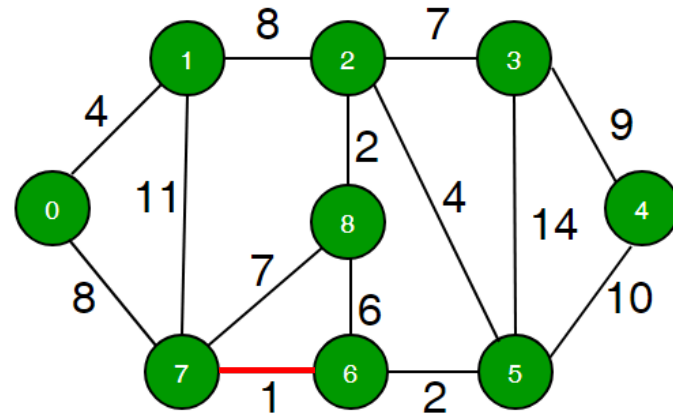


Menor aresta $6 \rightarrow 7$ com custo 1

Find(6) = 6

Find(7) = 7

Union(6,7) $\rightarrow$ 6 se torna o pai do 7



Union-Find

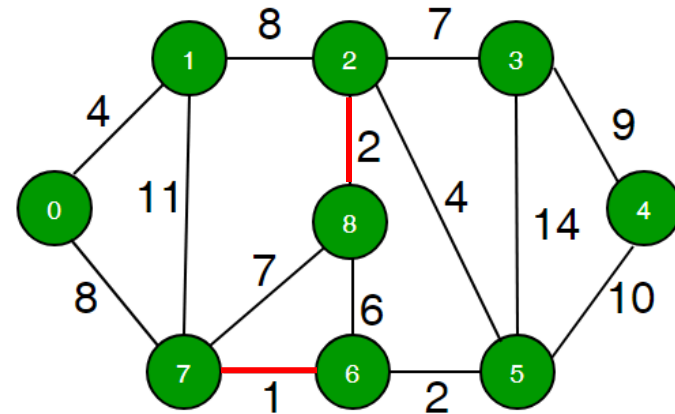


Menor aresta $2 \rightarrow 8$ com custo 2

Find(2) = 2

Find(8) = 8

Union(2,8) $\rightarrow$ 2 se torna o pai do 8



Union-Find

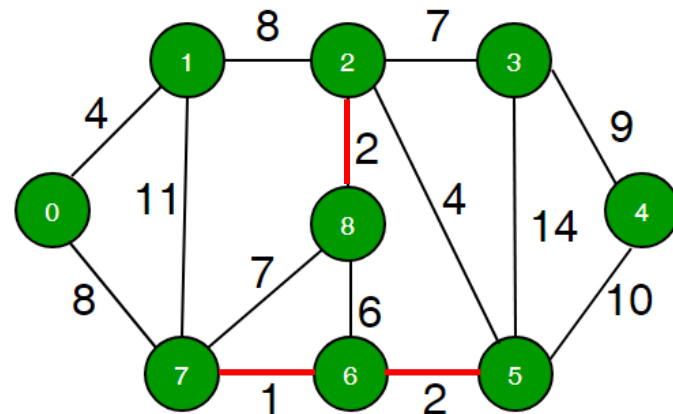


Menor aresta $5 \rightarrow 6$ com custo 2

Find(5) = 5

Find(6) = 6

Union(6,5) $\rightarrow$ 6 se torna o pai do 5



Union-Find

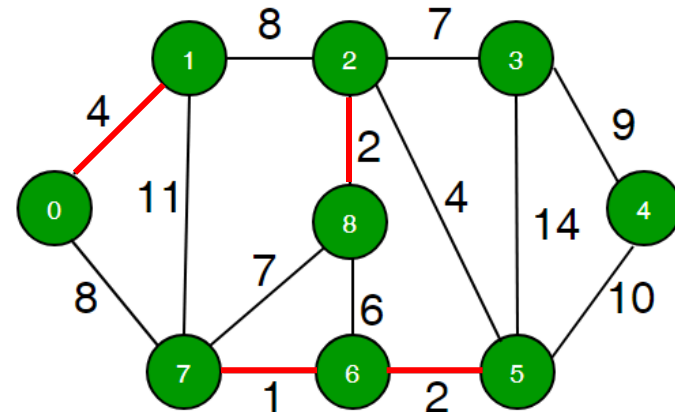


Menor aresta $0 \rightarrow 1$ com custo 4

Find(0) = 0

Find(1) = 1

Union(0,1) $\rightarrow$ 0 se torna o pai do 1



Union-Find

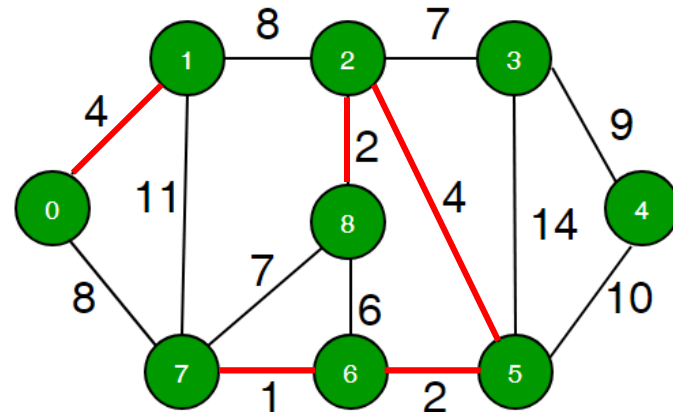


Menor aresta $2 \rightarrow 5$ com custo 4

Find(2) = 2

Find(5) = 6

Union(6,2) $\rightarrow$ 6 se torna o pai do 2



Union-Find

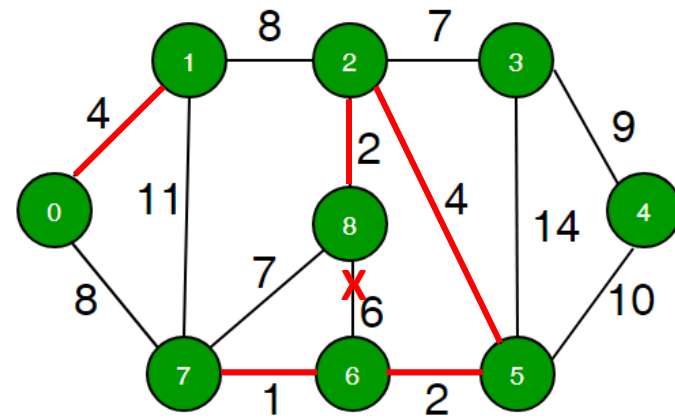


Menor aresta $6 \rightarrow 8$ com custo 6

$\text{Find}(6) = 6$

$\text{Find}(8) = 6$

Como pertencem ao mesmo grupo não
Faz a junção dos elementos



Union-Find

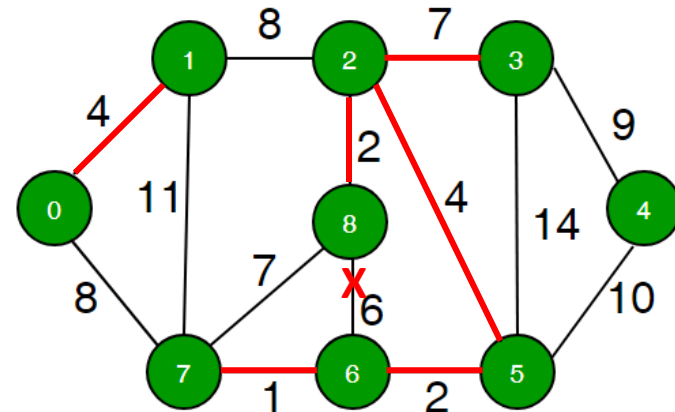


Menor aresta $2 \rightarrow 3$ com custo 7

Find(2) = 6

Find(3) = 3

Union(6,3) $\rightarrow$ 6 se torna o pai do 3



Union-Find

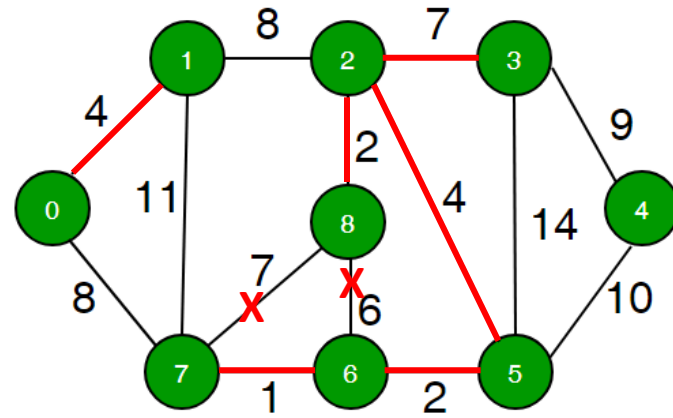


Menor aresta $7 \rightarrow 8$ com custo 7

Find(7) = 6

Find(8) = 6

Como pertencem ao mesmo grupo não
Faz a junção dos elementos



Union-Find

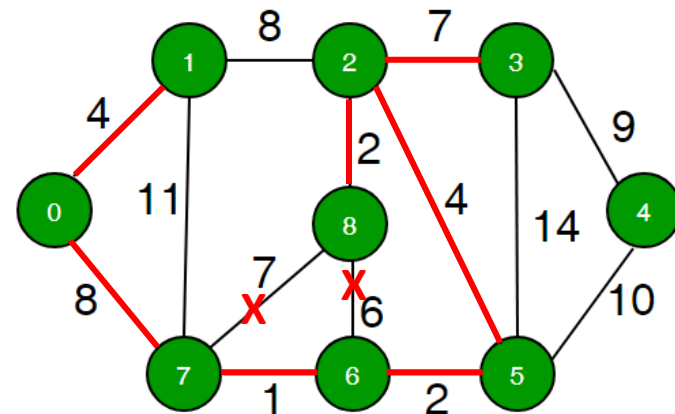


Menor aresta $0 \rightarrow 7$ com custo 8

$\text{Find}(0) = 0$

$\text{Find}(7) = 6$

$\text{Union}(6,0) \rightarrow 6$ se torna o pai do 0



Union-Find

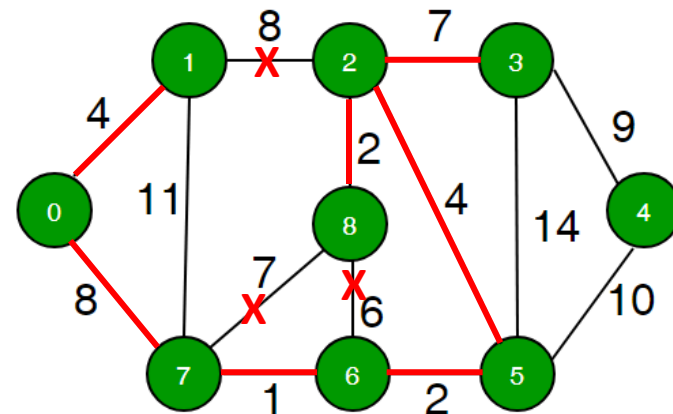


Menor aresta $1 \rightarrow 2$ com custo 8

$\text{Find}(1) = 6$

$\text{Find}(2) = 6$

Como pertencem ao mesmo grupo não
Faz a junção dos elementos



Union-Find

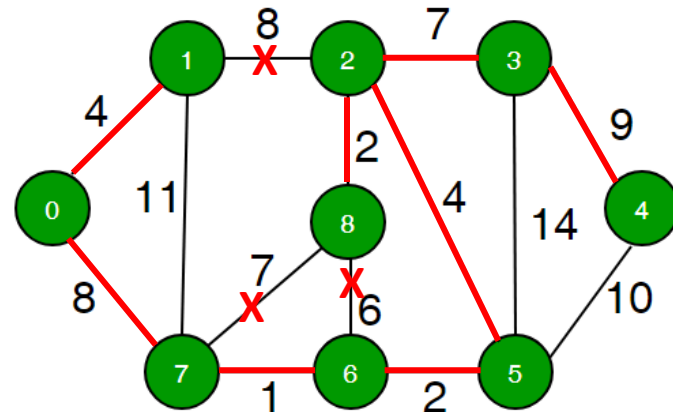


Menor aresta $3 \rightarrow 4$ com custo 9

Find(3) = 6

Find(4) = 4

Union(6,4) $\rightarrow$ 6 se torna o pai do 4

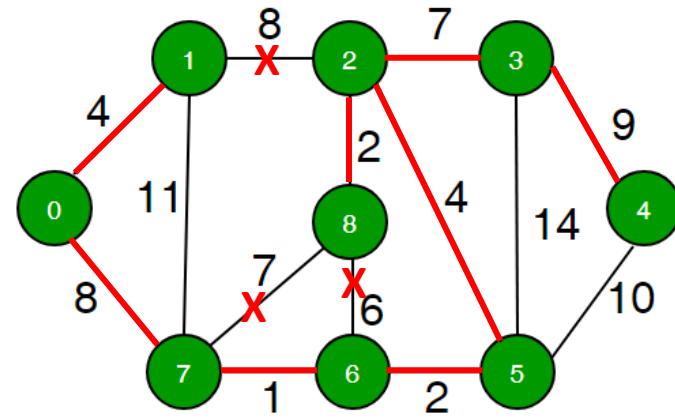


Union-Find



Todos os elementos pertencem ao mesmo conjunto, concluindo a execução do Kruskal

Não houve necessidade de testar as arestas $4 \rightarrow 5$, $1 \rightarrow 7$ e $3 \rightarrow 5$



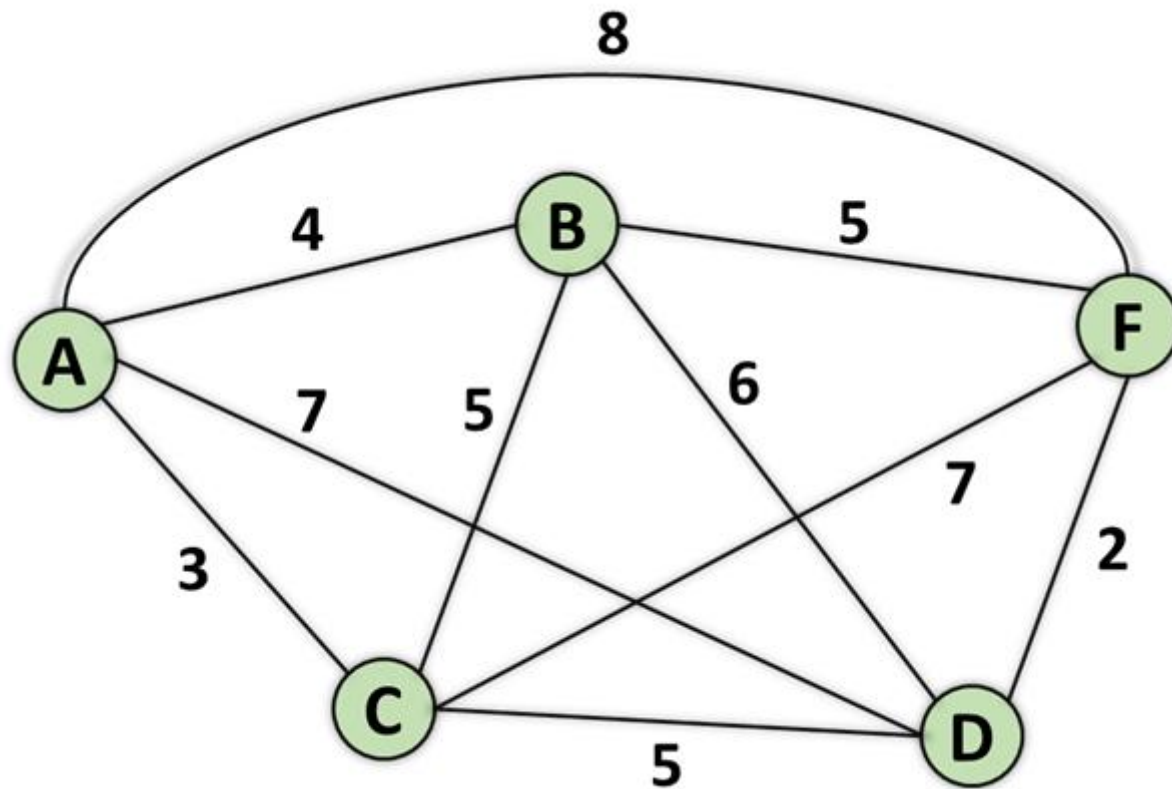
Prim

Entrada: G : grafo ponderado e não direcionado composto por V vértices e E arestas
 w : vetor contendo o peso W_i de cada uma das E arestas presentes no grafo G
Saída: conjunto das arestas selecionados para formar a AGM

```
1: for cada vértice  $v \in G.V$  do
2:    $W_i = \infty$ 
3:    $\pi_i = null$ 
4: end for
5:  $W_0 = 0$ 
6:  $Q = V[G]$ 
7: while ( $Q \neq \emptyset$ ) do
8:   Selecionar o vértice  $u$  com o menor custo ( $W_u$ )
9:   for cada vértice  $v \in G.Adj[u]$  do
10:    if ( $v \in Q$ ) && ( $W(u, v) < W_v$ ) then
11:       $W_v = W(u, v)$ 
12:       $\pi_v = u$ 
13:    end if
14:  end for
15: end while
16: return  $A$ 
```



Prim



Prim

Vértices	Iterações				
	1ª	2ª	3ª	4ª	5ª
A	0, -				
B	∞				
C	∞				
D	∞				
F	∞				



Prim

Vértices	Iterações				
	1ª	2ª	3ª	4ª	5ª
A	0, -				
B	∞	4, AB			
C	∞	3, AC			
D	∞	7, AD			
F	∞	8, AF			



Prim

Vértices	Iterações				
	1ª	2ª	3ª	4ª	5ª
A	0, -				
B	∞	4, AB	4, AB		
C	∞	3, AC			
D	∞	7, AD	5, CD		
F	∞	8, AF	7, CF		



Prim

Vértices	Iterações				
	1ª	2ª	3ª	4ª	5ª
A	0, -				
B	∞	4, AB	4, AB		
C	∞	3, AC			
D	∞	7, AD	5, CD	5, CD	
F	∞	8, AF	7, CF	5, BF	



Prim

Vértices	Iterações				
	1ª	2ª	3ª	4ª	5ª
A	0, -				
B	∞	4, AB	4, AB		
C	∞	3, AC			
D	∞	7, AD	5, CD	5, CD	
F	∞	8, AF	7, CF	5, BF	2, DF

